

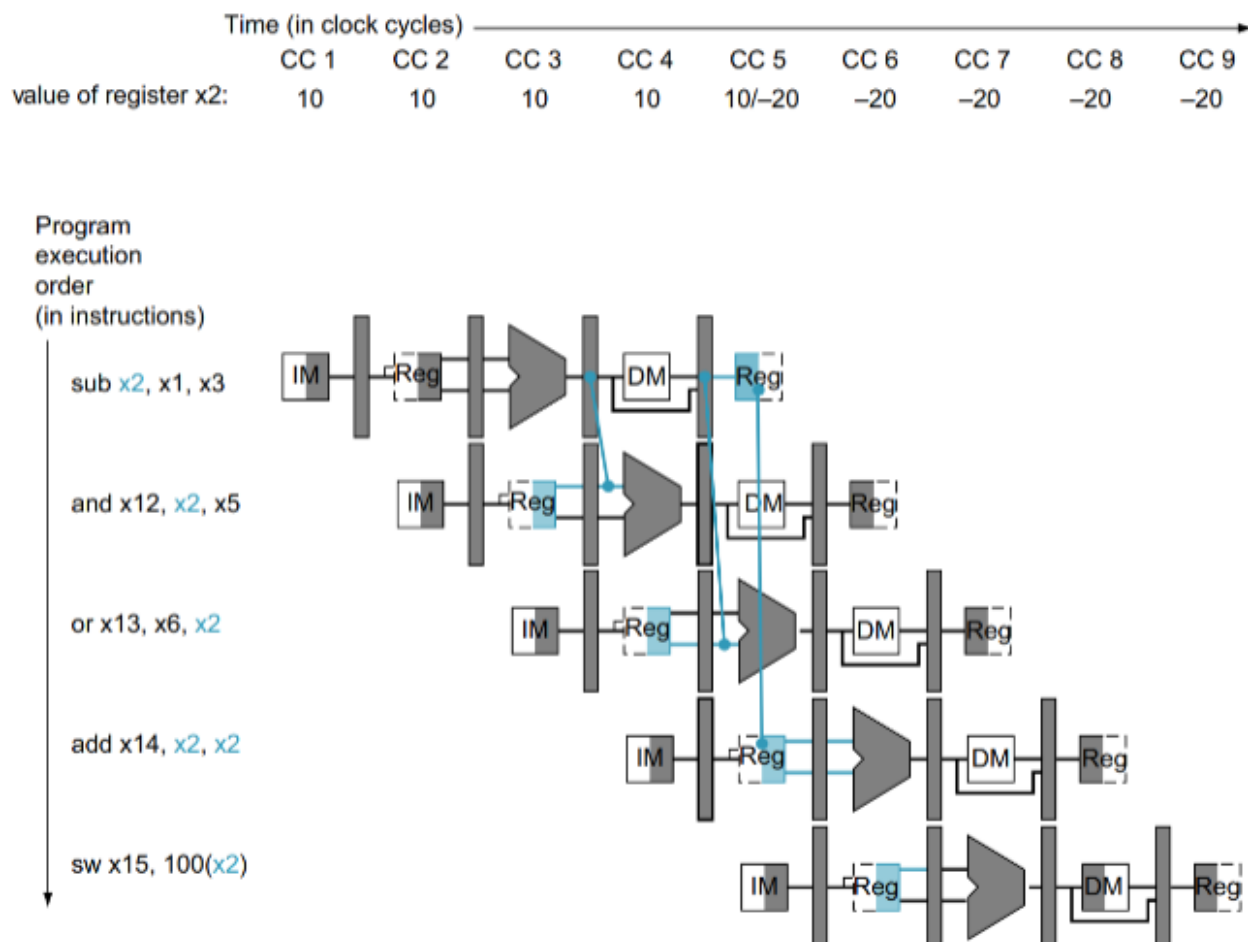
hardware approaches for dealing with data hazards

forwarding

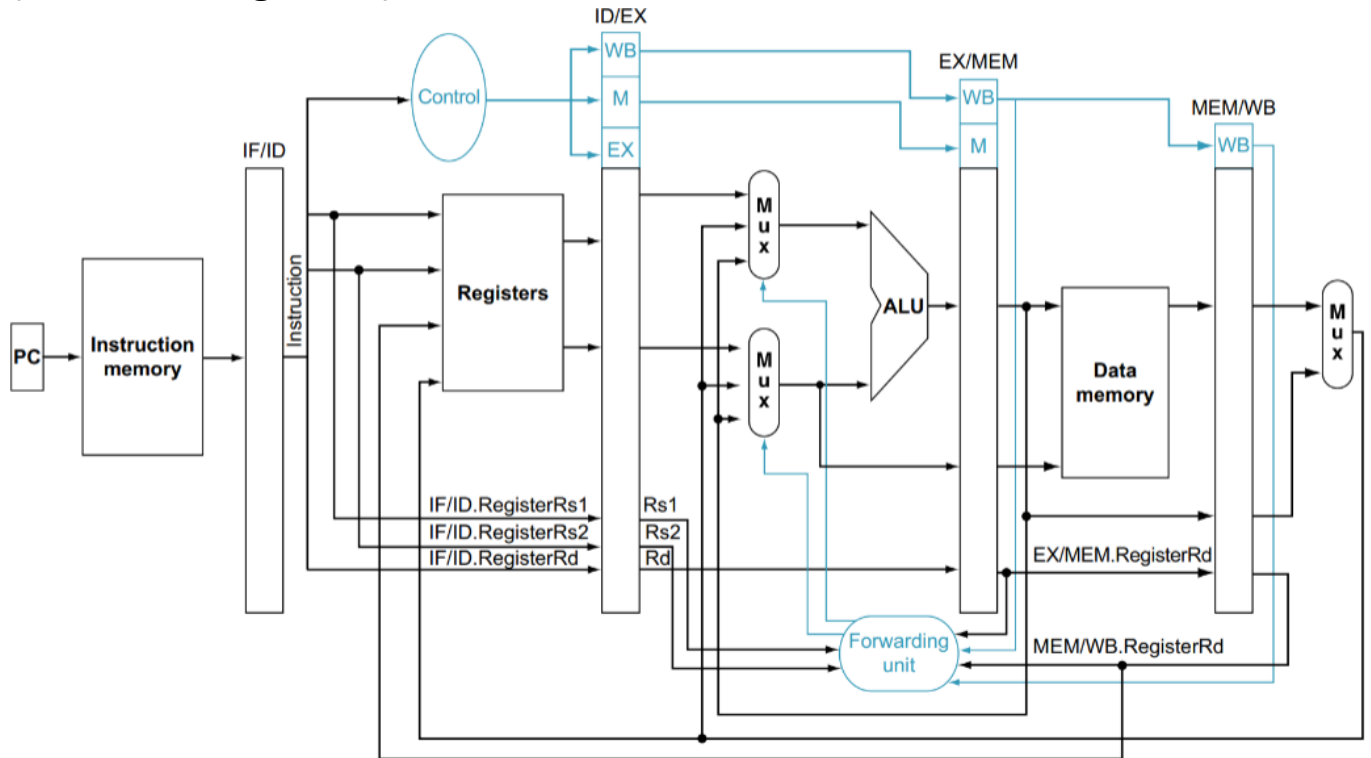
in some cases, the needed data *is* available, it just hasn't been written into the register file yet!

to deal with these hazards, **extra paths** can be added to the datapath to **forward** the required data from the **EX/MEM** or **MEM/WB** pipeline register

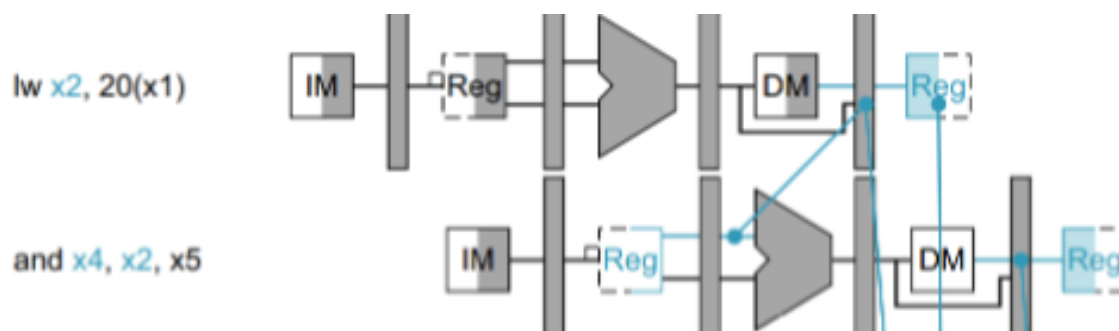
use of forwarding for previous (long) example
(from text Fig. 4.55)



implementation requires a **forwarding unit**, and **new multiplexors**, as in below *simplified* datapath (from text Fig. 4.58)



forwarding could be used to remove all data hazards for our pipeline **except** in the case of a “lw” followed immediately by an instruction operating on the value read from memory
(from portion of text Fig. 4.60)



stalling

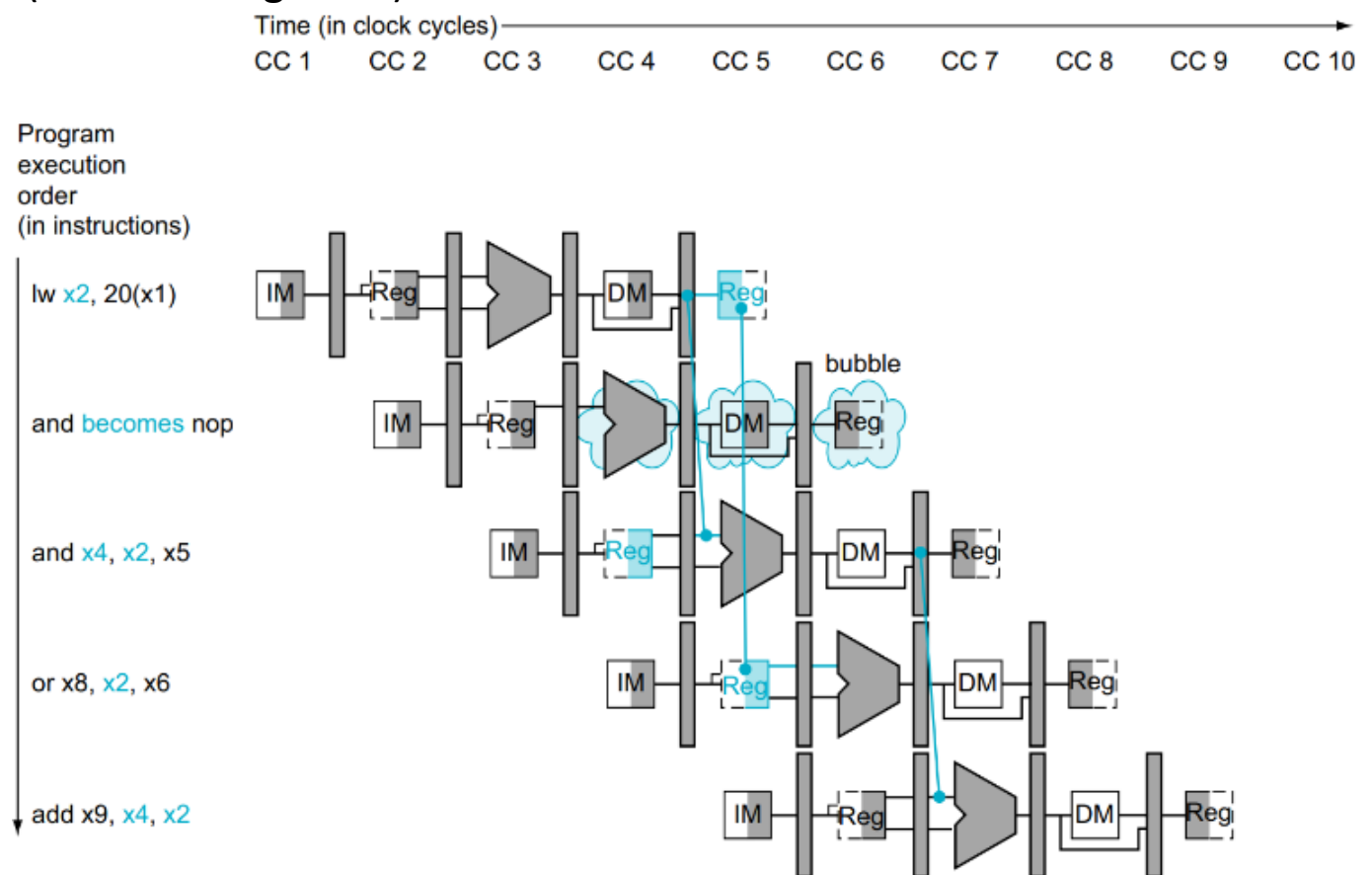
another hardware approach to data hazards

idea is to stop instructions in early pipeline stage(s)
from moving on until needed data is available

achieve by disabling writes into pipeline register(s)
and into the PC, creating a “bubble” in pipeline

bubble stages are given control line settings
equivalent to those of a “nop” instruction

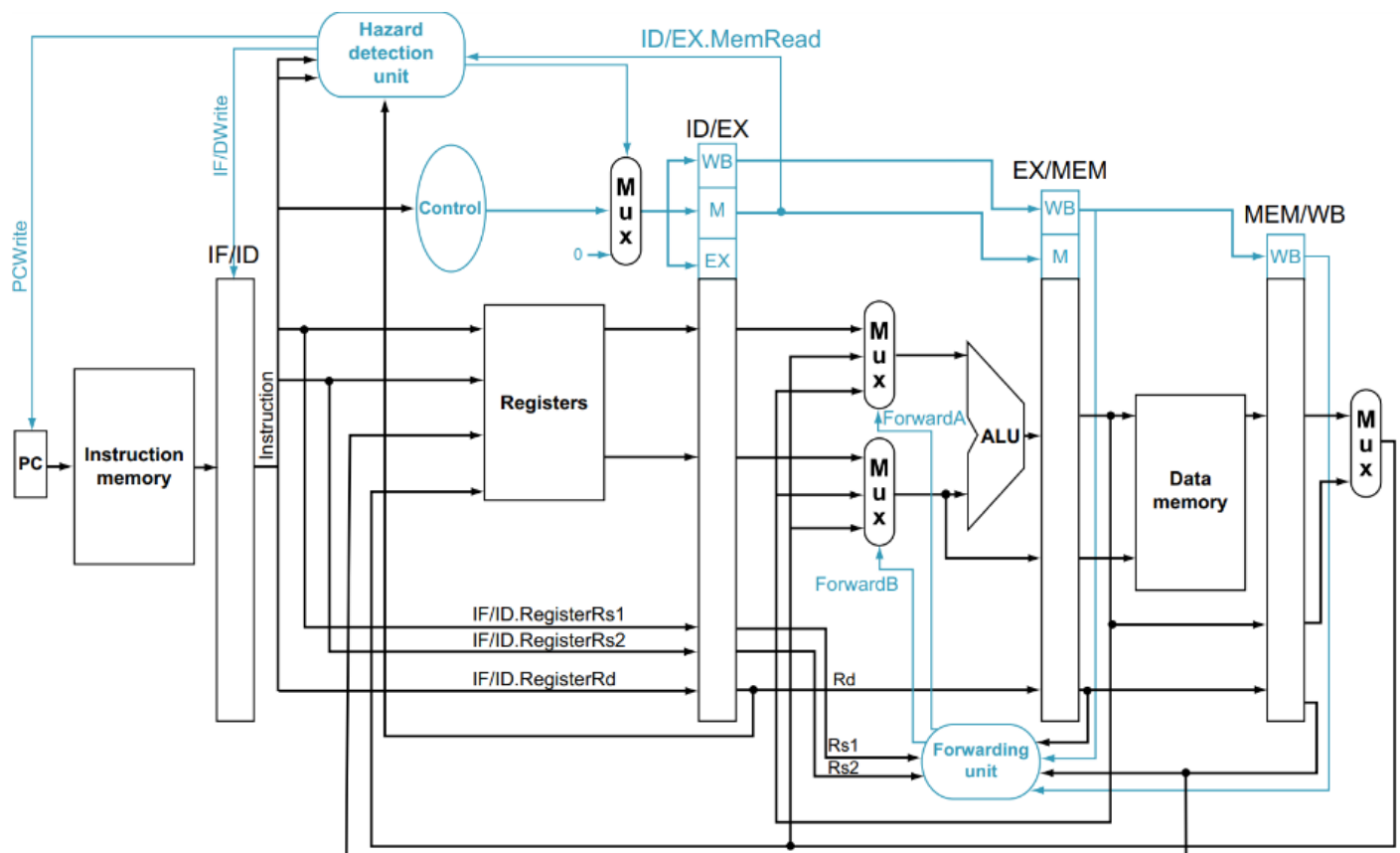
(from text Fig. 4.61)



stalling implementation requires a **hazard detection unit** that recognizes when a stall is needed

based on comparison of register numbers (similar idea as with forwarding)

simplified picture of implementation when forwarding is also used, and so stalling is needed only for case of "lw" followed by an instruction using the value read from memory (from text Fig. 4.62)



control hazards

arise when execute conditional branch instructions

major concern since in most programs branches occur frequently:

- conditional branches have been reported to comprise about **20% of executed instructions** (less in numerically-intensive applications)

for example, consider following instruction sequence:

```
    addi s3, s3, 1
    beq s0, zero, skip
    add s4, s1, zero
skip: lw s2, 0(s1)
```

with the pipelined datapath we have designed, the branch decision for the “**beq**” won't occur until its **4th step** of execution

... but if pipelining is to proceed normally, we need to fetch the next instruction when the “beq” instruction is in its **2nd step** of execution!

conditional instructions

one approach to dealing with branch hazards is to (when possible) remove the branch!

for this purpose, add **conditional instructions** to the instruction set

RISC-V conditional instructions “**czero.eqz**” and “**czero.nez**” are in the ‘Zicond’ extension; using “**czero.nez**” one could remove the branch in the previous example, eliminating the hazard:

```
addi s3, s3, 1
sub s4, s4, s1
czero.nez s4, s4, s0  # if s0 ≠ 0, s4 ← 0; else s4 ← s4
add s4, s4, s1        # tricky ...
lw s2, 0(s1)
```

faster branches

a second approach is to **reduce the impact** of branch hazards by making the branch decision in an earlier pipeline stage

- easy part is earlier calculation of branch target address, harder part is earlier comparison of operands (*why?*)